LAPORAN TUGAS PEMROGRAMAN WEB

COVER

Judul Website	**Stok Dapur** — Aplikasi Manajemen Stok Bahan Baku Dapur
Nama Anggota	*(isi nama anggota kelompok)*
NPM	*(isi NPM masing-masing)*
Mata Kuliah	Pemrograman Web — Semester 5

Tools yang Digunakan

Kategori	Tools
Bahasa Pemrograman	JavaScript (ES6+)
Frontend	HTML5, CSS3, Vanilla JavaScript (tanpa framework)
Backend	Node.js + Express.js
Database	MongoDB Atlas (cloud)
Deployment Frontend	GitHub Pages
Deployment Backend	Vercel (Serverless Functions)
Font	Google Fonts (Bricolage Grotesque, IBM Plex Sans, IBM Plex Mono)
Version Control	Git + GitHub
Code Editor	Visual Studio Code
Package Manager	npm

DAFTAR ISI

1. Penjelasan Semua Fitur Web Beserta Output
    - 1.1 Halaman Login
    - 1.2 Tema Terang & Gelap (Dark Mode)
    - 1.3 Tab Rak — Daftar Bahan
    - 1.4 Ringkasan Stok
    - 1.5 Tambah Bahan Baru
    - 1.6 Edit Bahan
    - 1.7 Hapus Bahan (dengan Konfirmasi)
    - 1.8 Tab Catat — Catat Transaksi Stok Masuk/Keluar
    - 1.9 Tab Jurnal — Riwayat Transaksi
    - 1.10 Bon Belanja (Sidebar)
    - 1.11 Toast Notifikasi
    - 1.12 Logout
2. Konfigurasi Website (Backend & Frontend)
    - 2.1 Arsitektur Aplikasi
    - 2.2 Konfigurasi Frontend
    - 2.3 Konfigurasi Backend
    - 2.4 Konfigurasi Database
    - 2.5 Konfigurasi Deployment
3. Penjelasan Source Code
    - 3.1 Struktur Folder
    - 3.2 Frontend — index.html
    - 3.3 Frontend — app.js
    - 3.4 Frontend — api.js
    - 3.5 Frontend — ui.js
    - 3.6 Frontend — style.css
    - 3.7 Backend — server.js
4. Daftar Isi *(sudah di atas)*

1. PENJELASAN SEMUA FITUR WEB BESERTA OUTPUT

> NOTE
>
> Setiap fitur di bawah ini perlu dilengkapi **screenshot** tampilan output dari website. Ambil screenshot langsung dari browser saat fitur berjalan, lalu tempelkan di bawah penjelasan masing-masing fitur.

1.1 Halaman Login

Deskripsi: Halaman pertama yang ditampilkan saat pengguna membuka website. Pengguna harus memasukkan **username** dan **password** untuk masuk ke aplikasi. Sistem autentikasi menggunakan metode **Basic Auth** — username dan password dikirim ke backend untuk dicocokkan dengan data di database MongoDB.

Cara Kerja:

- Pengguna mengisi field username dan password lalu klik tombol "Masuk"
- Frontend mengirim `POST /api/auth/masuk` ke backend
- Backend mencocokkan dengan collection `users` di MongoDB
- Jika cocok → token (base64 dari username:password) disimpan di `localStorage` dan halaman utama ditampilkan
- Jika salah → muncul pesan error merah di bawah form: "*Username atau password salah*"

Output: *(tambahkan screenshot halaman login di sini)*

1.2 Tema Terang & Gelap (Dark Mode)

Deskripsi: Pengguna bisa mengganti tema tampilan antara **Mode Terang** (default) dan **Mode Gelap** dengan menekan tombol toggle "Mode gelap" / "Mode terang". Tombol ini tersedia di halaman login maupun di halaman utama aplikasi.

Cara Kerja:

- Klik tombol "Mode gelap" → atribut `data-theme="dark"` ditambahkan ke elemen `<html>`
- CSS menggunakan **CSS Custom Properties** (variabel) yang berubah otomatis sesuai tema
- Pilihan tema disimpan di `localStorage` dengan key `stockdapur-tema` sehingga saat halaman dibuka ulang, tema yang terakhir dipilih langsung diterapkan

Output: *(tambahkan screenshot tema terang vs tema gelap)*

1.3 Tab Rak — Daftar Bahan

Deskripsi: Tab utama yang menampilkan semua bahan baku dalam bentuk **kartu grid** (4 kolom di desktop, responsif ke 1 kolom di mobile). Setiap kartu bahan menampilkan:

- **Badge status** → warna hijau (Aman), kuning (Hampir menipis), atau merah (Menipis)
- **Nama bahan** (contoh: Tahu putih)
- **Jumlah stok saat ini** dengan satuan (contoh: 5 kg)
- **Progress bar** → visualisasi level stok relatif terhadap batas minimum
- **Info tambahan** → stok minimum dan harga per satuan
- **Tombol aksi** → Edit (✏️), Hapus (🗑️), dan "Catat stok"

Logika Status:

- **Menipis** (merah): stok < stok minimum
- **Hampir menipis** (kuning): stok >= stok minimum, tapi < 1.5× stok minimum
- **Aman** (hijau): stok >= 1.5× stok minimum

Output: *(tambahkan screenshot halaman Rak dengan kartu-kartu bahan)*

1.4 Ringkasan Stok

Deskripsi: Bar ringkasan di bagian atas halaman Rak yang menampilkan 3 informasi penting:

1. **Total jumlah bahan** yang terdaftar
2. **Jumlah bahan yang menipis** (ditandai warna merah)
3. **Nilai total stok** — dihitung dari stok × harga satuan semua bahan (dalam format Rupiah)

Output: *(tambahkan screenshot bar ringkasan)*

1.5 Tambah Bahan Baru

Deskripsi: Form untuk menambahkan bahan baru ke dalam sistem. Form ini berada di bawah grid kartu dan bisa dibuka/ditutup dengan klik pada "Tambah bahan baru" (menggunakan elemen `<details>`).

Field yang harus diisi:

Field	Keterangan
Nama bahan	Nama bahan baku (contoh: Tahu putih)
Satuan	Satuan yang digunakan (contoh: kg, liter, pcs)
Stok awal	Jumlah stok pertama kali ditambahkan
Stok minimum	Batas minimum sebelum bahan dianggap menipis
Harga satuan	Harga per satuan dalam Rupiah

Cara Kerja:

- Frontend mengirim `POST /api/stok` ke backend
- Backend menyimpan ke collection `bahan` di MongoDB
- Setelah berhasil, kartu bahan baru langsung muncul di grid tanpa reload halaman

Output: *(tambahkan screenshot form tambah bahan)*

1.6 Edit Bahan

Deskripsi: Fitur untuk mengubah data bahan yang sudah ada. Dipicu dengan klik tombol ✏️ di kartu bahan.

Cara Kerja:

- Saat tombol edit ditekan, form "Tambah bahan baru" otomatis terbuka dan terisi data bahan tersebut
- Field **stok awal dinonaktifkan** (disabled) karena perubahan stok hanya boleh dilakukan lewat fitur Catat
- Tombol berubah menjadi "Simpan perubahan"
- Frontend mengirim `PUT /api/stok/:id` ke backend
- Halaman scroll otomatis ke posisi form

Output: *(tambahkan screenshot form dalam mode edit)*

1.7 Hapus Bahan (dengan Konfirmasi)

Deskripsi: Fitur untuk menghapus bahan dari sistem. Dipicu dengan klik tombol 🗑️ di kartu bahan.

Cara Kerja:

- Saat tombol hapus ditekan, muncul **modal konfirmasi** bertuliskan "*Yakin ingin menghapus [nama bahan]?*"
- Pengguna bisa memilih "Batal" atau "Hapus"
- Jika dikonfirmasi, frontend mengirim `DELETE /api/stok/:id` ke backend
- Bahan langsung hilang dari tampilan tanpa reload
- Toast notifikasi merah muncul: "*Bahan berhasil dihapus*"

Output: *(tambahkan screenshot modal konfirmasi hapus)*

1.8 Tab Catat — Catat Transaksi Stok Masuk/Keluar

Deskripsi: Form untuk mencatat transaksi stok bahan, bisa berupa **stok masuk** (beli/terima bahan) atau **stok keluar** (pakai/keluarkan bahan).

Komponen:

- **Toggle Masuk/Keluar** — memilih jenis transaksi
- **Dropdown Bahan** — berisi daftar semua bahan beserta stok saat ini
- **Field Jumlah** — labelnya berubah dinamis sesuai satuan bahan yang dipilih (contoh: "Jumlah masuk (kg)")
- **Field Catatan** — opsional, untuk keterangan (contoh: "beli dari pasar")
- **Daftar 10 transaksi terakhir** — ditampilkan di bawah form

Validasi:

- Jumlah harus lebih dari 0
- Jika stok keluar > stok saat ini → muncul error "*Stok tidak cukup*"

Cara Kerja:

- Frontend mengirim `POST /api/stok/:id/catat` ke backend
- Backend menghitung stok baru dan menyimpan ke database
- Frontend memperbarui data lokal dan menampilkan transaksi di riwayat

Output: *(tambahkan screenshot tab Catat dengan form dan riwayat)*

1.9 Tab Jurnal — Riwayat Transaksi

Deskripsi: Menampilkan **seluruh riwayat transaksi** yang pernah dicatat selama sesi berlangsung, diurutkan dari yang paling baru.

Setiap baris transaksi menampilkan:

- Waktu transaksi (format: "2 Nov, 21.30")
- Nama bahan
- Jumlah dengan tanda **+** (hijau) untuk masuk atau **–** (merah) untuk keluar
- Catatan (jika ada)

Perbedaan dengan Tab Catat:

- Tab Catat hanya menampilkan **10 transaksi terakhir**
- Tab Jurnal menampilkan **semua transaksi** tanpa batas

Output: *(tambahkan screenshot tab Jurnal)*

1.10 Bon Belanja (Sidebar)

Deskripsi: Sidebar di sisi kanan halaman yang **otomatis menghitung bahan-bahan yang stoknya menipis** dan menampilkannya sebagai daftar belanja lengkap dengan estimasi biaya.

Isi Bon:

- Tanggal hari ini
- Daftar bahan yang perlu dibeli (stok < minimum)
- Jumlah yang perlu dibeli = stok minimum − stok saat ini
- Estimasi biaya per bahan = jumlah perlu beli × harga satuan
- **Total estimasi biaya** keseluruhan
- Jika semua stok aman → muncul pesan "*Semua stok aman* 👍"

Output: *(tambahkan screenshot bon belanja saat ada bahan menipis, dan saat semua aman)*

1.11 Toast Notifikasi

Deskripsi: Notifikasi kecil yang muncul di bagian bawah layar setiap ada aksi berhasil atau gagal. Toast muncul dengan animasi slide-up, bertahan 2.5 detik, lalu menghilang otomatis.

Contoh pesan:

- ✅ "*Bahan berhasil ditambahkan*"
- ✅ "*Bahan berhasil diubah*"
- ✅ "*5 kg Tahu putih berhasil ditambahkan*"
- 🔴 "*Bahan berhasil dihapus*" (warna merah)

- ✅ *"Data berhasil dimuat ulang"*

Output: *(tambahkan screenshot toast notifikasi)*

1.12 Logout

Deskripsi: Fitur untuk keluar dari aplikasi. Tombol "Keluar" berada di pojok kanan atas di sebelah nama pengguna.

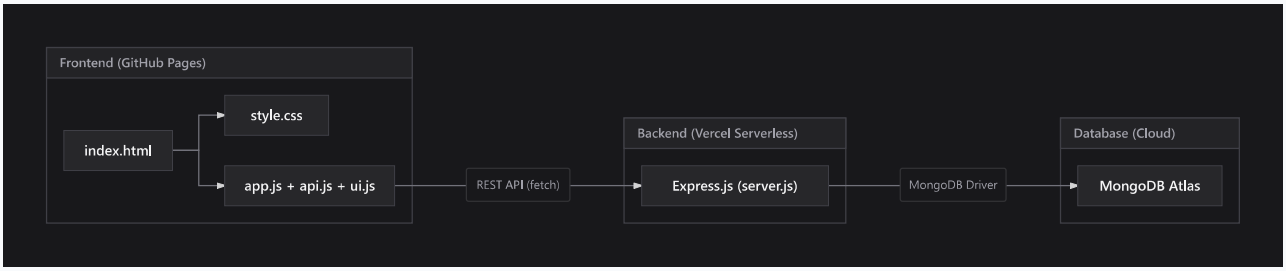
Cara Kerja:

- Token dihapus dari `localStorage`
- Semua state (data bahan, riwayat) dikosongkan
- Halaman utama disembunyikan dan halaman login ditampilkan kembali
- Form login direset

Output: *(tambahkan screenshot link keluar)*

2. KONFIGURASI WEBSITE (BACKEND & FRONTEND)

2.1 Arsitektur Aplikasi



Aplikasi ini menggunakan arsitektur **client-server** dengan pemisahan penuh antara frontend dan backend:

Komponen	Teknologi	Hosting
Frontend	HTML + CSS + Vanilla JS	GitHub Pages (`ilhamrezapratama.github.io`)
Backend	Node.js + Express.js	Vercel Serverless (`stock-dapur.vercel.app`)
Database	MongoDB Atlas	MongoDB Cloud (`cluster0.zcktnrt.mongodb.net`)

2.2 Konfigurasi Frontend

Tidak menggunakan framework. Frontend dibangun sepenuhnya dengan:

- **HTML5** — struktur halaman single-page (SPA tanpa routing library)
- **CSS3** — styling dengan CSS Custom Properties (variabel) untuk mendukung tema terang/gelap
- **JavaScript ES6+** — menggunakan fitur `import/export` (ES Modules), `async/await`, template literals, arrow functions
- **Google Fonts** — 3 font: Bricolage Grotesque (heading), IBM Plex Sans (body), IBM Plex Mono (monospace/bon)

Struktur file frontend:

```
frontend/
├── css/
│   └── style.css       (228 baris — semua styling + responsive)
└── js/
    ├── app.js          (326 baris — logika utama + event listener)
    ├── api.js          (88 baris — komunikasi ke backend via fetch)
    └── ui.js           (156 baris — render tampilan DOM)
```

Konfigurasi API:

```
javascript
// api.js — Base URL backend
const BASE = 'https://stock-dapur.vercel.app/api';
```

2.3 Konfigurasi Backend

Framework: Express.js v4.18.2 di atas Node.js

Dependencies (`package.json`):

```json
{
  "dependencies": {
    "cors": "^2.8.5",
    "dotenv": "^16.0.0",
    "express": "^4.18.2",
    "mongodb": "^6.3.0"
  }
}
```

Package	Fungsi
`express`	Web framework untuk membuat API server
`mongodb`	Driver resmi MongoDB untuk Node.js (tanpa Mongoose)
`dotenv`	Membaca variabel environment dari file `.env`
`cors`	(tersedia tapi CORS dihandle manual di code)

Daftar API Endpoint:

Method	Endpoint	Fungsi	Auth
`POST`	`/api/auth/masuk`	Login pengguna	❌
`POST`	`/api/auth/keluar`	Logout pengguna	❌
`GET`	`/api/auth/saya`	Cek sesi/token aktif	✅
`GET`	`/api/stok`	Ambil semua data bahan	✅
`POST`	`/api/stok`	Tambah bahan baru	✅
`PUT`	`/api/stok/:id`	Edit data bahan	✅
`DELETE`	`/api/stok/:id`	Hapus bahan	✅
`POST`	`/api/stok/:id/catat`	Catat transaksi stok	✅

Sistem Autentikasi:

- Menggunakan **Basic Auth** (stateless)
- Token = `Base64(username:password)`
- Token dikirim di header `Authorization: Basic <token>` setiap request

- Middleware `harusLogin` memverifikasi token dengan query ke database

2.4 Konfigurasi Database

Database: MongoDB Atlas (cloud-hosted)

Parameter	Nilai
Provider	MongoDB Atlas
Cluster	`cluster0.zcktnrt.mongodb.net`
Database Name	`stockdapur`
Driver	Native MongoDB Node.js Driver v6.3.0

Collections:

Collection	Keterangan	Contoh Document
`users`	Data pengguna	`{ username, password, nama, role }`
`bahan`	Data bahan baku	`{ nama_bahan, stok, satuan, min_stok, harga_satuan }`

Koneksi: Menggunakan connection pooling — koneksi database di-cache agar tidak dibuat ulang setiap request (penting untuk serverless environment).

2.5 Konfigurasi Deployment

Frontend → GitHub Pages

- Repository GitHub di-push, lalu GitHub Pages melayani file `index.html` sebagai halaman statis
- URL: `https://ilhamrezapratama.github.io/StockDapur-main/`

Backend → Vercel Serverless

File `vercel.json`:

```
json
{
  "version": 2,
  "builds": [
    { "src": "server.js", "use": "@vercel/node" }
  ],
  "routes": [
    {
      "src": "/(.*)",
      "methods": ["OPTIONS"],
      "headers": {
        "Access-Control-Allow-Origin": "https://ilhamrezapratama.github.io",
        "Access-Control-Allow-Credentials": "true",
        "Access-Control-Allow-Methods": "GET,POST,PUT,DELETE,OPTIONS",
        "Access-Control-Allow-Headers": "Content-Type,Authorization"
      },
      "status": 200,
      "continue": false
    },
    { "src": "/(.*)", "dest": "server.js" }
  ]
}
```

- `server.js` dijalankan sebagai Vercel Serverless Function menggunakan `@vercel/node`
- CORS di-set untuk hanya menerima request dari domain GitHub Pages
- URL: `https://stock-dapur.vercel.app`

3. PENJELASAN SOURCE CODE

3.1 Struktur Folder

```
StockDapur-main/
├── .gitignore              → File/folder yang diabaikan Git
├── index.html              → Halaman utama (entry point)
├── backend/
│   ├── server.js           → Seluruh backend (Express + routes + DB)
│   ├── package.json        → Dependencies backend
│   ├── package-lock.json   → Lock file npm
│   ├── vercel.json         → Konfigurasi deploy ke Vercel
│   └── .env                → Variabel environment (MONGO_URI)
└── frontend/
    ├── js/
    │   ├── app.js           → Controller utama (state + event + logic)
    │   ├── api.js           → Fungsi fetch ke backend
    │   └── ui.js            → Fungsi render tampilan
    └── css/
        └── style.css        → Seluruh styling + tema + responsive
```

3.2 Frontend — `index.html` (219 baris)

File HTML utama yang menjadi kerangka seluruh aplikasi. Menggunakan pendekatan **Single Page Application (SPA)** — semua halaman ada di satu file HTML dan ditampilkan/disembunyikan oleh JavaScript.

Struktur utama:

```
<body>
├── #login-page         → Halaman login (ditampilkan pertama)
│   └── form#form-login → Form username + password
│
├── #app                → Halaman utama (hidden, muncul setelah login)
│   ├── <header>        → Topbar (judul, user, theme toggle, tabs)
│   └── .layout         → Layout 2 kolom
│       ├── .content    → Konten utama
│       │   ├── #tab-rak    → Grid kartu bahan + form tambah
│       │   ├── #tab-catat  → Form catat + 10 riwayat terakhir
│       │   └── #tab-jurnal → Seluruh riwayat transaksi
│       └── .bon        → Sidebar bon belanja
│
├── #confirm-overlay    → Modal konfirmasi hapus
├── #toast              → Toast notifikasi
└── <script type="module" src="frontend/js/app.js">
```

Poin penting:

- Menggunakan `<script type="module">` agar bisa menggunakan `import/export` antar file JS
- Halaman login ditampilkan dengan `display:flex`, halaman app dengan `display:none` (diubah oleh JS)
- Tab menggunakan class `.active` untuk menampilkan/menyembunyikan panel

3.3 Frontend — `app.js` (326 baris)

File JavaScript utama yang mengatur **seluruh alur kerja aplikasi**. Bertanggung jawab atas:

1. State Management (baris 17-23):

```
javascript
let penggunaAktif = null;   // Info pengguna yang login
let daftarBahan   = [];     // Semua data bahan dari DB
let riwayat       = [];     // Transaksi yang dicatat sesi ini
let tabAktif      = 'rak';  // Tab yang sedang aktif
let tipeCapatan   = 'masuk';// Mode masuk/keluar
let idSedangEdit  = null;   // ID bahan yang sedang diedit
let idAkanHapus   = null;   // ID bahan yang akan dihapus
```

2. Fungsi-fungsi Utama:

Fungsi	Baris	Tugas
`toast(pesan, tipe)`	32-37	Tampilkan notifikasi toast
`setTheme(tema)`	44-52	Ganti tema terang/gelap
`render()`	58-63	Render ulang tampilan sesuai tab aktif
`pindahTab(nama)`	70-75	Pindah antar tab Rak/Catat/Jurnal
`prosesLogin(e)`	82-92	Proses form login
`tampilkanApp(pengguna)`	97-106	Tampilkan halaman utama setelah login
`prosesLogout()`	112-120	Proses logout + reset state
`simpanBahan(e)`	127-151	Simpan bahan baru atau perubahan edit
`window.startEdit(id)`	159-174	Aktifkan mode edit bahan
`resetFormBahan()`	178-184	Reset form ke kondisi awal
`window.confirmDel(id)`	192-198	Tampilkan modal konfirmasi hapus
`hapusBahan()`	204-213	Proses hapus bahan setelah dikonfirmasi
`window.catatStok(id)`	221-226	Pindah ke tab Catat + pilih bahan
`prosesCatat(e)`	235-264	Proses form catat transaksi
`setTipeCapatan(tipe)`	269-275	Toggle antara masuk/keluar
`resetData()`	288-296	Muat ulang data dari database

3. Inisialisasi (baris 304-326):

- Terapkan tema dari `localStorage`
- Cek sesi login yang masih aktif via `cekSesiApi()`
- Daftarkan semua event listener untuk form, tombol, tab, dll

3.4 Frontend — `api.js` (88 baris)

File ini berisi semua fungsi yang berkomunikasi dengan backend via `fetch()`. Semua request dibungkus dalam fungsi async yang mengembalikan JSON.

Konfigurasi:

```
javascript
const BASE = 'https://stock-dapur.vercel.app/api';
let authToken = localStorage.getItem('stockdapur-token') || '';
```

Helper function `opt(method, body)`:

- Membuat opsi request secara seragam
- Menyertakan header `Authorization: Basic <token>` jika token tersedia
- Menyertakan `Content-Type: application/json`

Daftar fungsi:

Fungsi	Method	Endpoint	Keterangan
`loginApi(username, katasandi)`	POST	`/auth/masuk`	Login, simpan token
`logoutApi()`	—	—	Hapus token dari localStorage
`cekSesiApi()`	GET	`/auth/saya`	Verifikasi token masih valid
`getBahanApi()`	GET	`/stok`	Ambil semua bahan
`tambahBahanApi(data)`	POST	`/stok`	Tambah bahan baru
`ubahBahanApi(id, data)`	PUT	`/stok/:id`	Update data bahan
`hapusBahanApi(id)`	DELETE	`/stok/:id`	Hapus bahan
`catatApi(id, data)`	POST	`/stok/:id/catat`	Catat transaksi

3.5 Frontend — `ui.js` (156 baris)

File ini berisi semua fungsi yang merender tampilan ke DOM. Dipisahkan dari logika di `app.js` agar masing-masing file punya tanggung jawab yang jelas.

Fungsi Pembantu (Helper):

Fungsi	Contoh Output	Keterangan
`formatRp(38000)`	`Rp38.000`	Format angka ke Rupiah
`formatJumlah(1500.5)`	`1.500,5`	Format angka dengan pemisah ribuan
`formatWaktu(iso)`	`2 Nov, 21.30`	Format waktu dari ISO string
`hariIni()`	`Minggu, 2 Nov 2026`	Tanggal hari ini lengkap

Fungsi `getStatus(bahan)` — Logika status stok:

```
javascript
if (stok < min_stok)          → { label: 'Menipis',        cls: 'red'    }
if (stok < min_stok × 1.5)    → { label: 'Hampir menipis', cls: 'yellow' }
else                          → { label: 'Aman',           cls: 'green'  }
```

Fungsi Render:

Fungsi	Target DOM	Keterangan
`renderRak(bahanList)`	`#summary-bar` + `#bahan-grid`	Render ringkasan + kartu bahan
`renderBon(bahanList)`	`#bon-belanja`	Render sidebar bon belanja
`renderCatat(bahanList, riwayat, tipe)`	`#catat-bahan` + `#recent-entries`	Render dropdown + 10 riwayat terakhir
`renderJurnal(riwayat)`	`#jurnal-entries`	Render seluruh riwayat
`entryHTML(transaksi)`	—	Template HTML untuk satu baris riwayat

3.6 Frontend — `style.css` (228 baris)

File CSS yang mengatur seluruh tampilan aplikasi.

Fitur CSS yang digunakan:

Fitur	Keterangan
CSS Custom Properties	Variabel warna dan font yang berubah sesuai tema
CSS Grid	Layout 2 kolom (konten + bon) dan grid 4 kolom untuk kartu bahan
Flexbox	Topbar, kartu, tombol, ringkasan
Media Queries	Responsive di 3 breakpoint: 1100px, 860px, 540px
Transitions	Animasi hover, toggle, toast slide-up
Sticky positioning	Bon belanja tetap terlihat saat scroll

Skema Tema:

- **Tema Terang:** Background `#F3F5F2`, kartu putih, aksen hijau `#47785D`
- **Tema Gelap:** Background `#000000`, kartu hitam, aksen hijau lebih terang `#609474`

Responsive Breakpoints:

- **> 1100px** → 4 kolom kartu, layout 2 kolom
- **860px - 1100px** → 3 kolom kartu
- **540px - 860px** → 2 kolom kartu, bon pindah ke atas
- **< 540px** → 1 kolom kartu, topbar vertikal, form field stack

3.7 Backend — `server.js` (159 baris)

Seluruh backend ditulis dalam **satu file** tanpa pemisahan ke folder routes/models/controllers. Menggunakan Express.js dengan native MongoDB driver (bukan Mongoose).

Struktur code:

```
server.js
├── Import & Setup            (baris 1-12)
├── getDB()                   (baris 18-24)  → Koneksi MongoDB dengan caching
├── CORS Middleware           (baris 28-35)  → Manual CORS untuk GitHub Pages
├── harusLogin() Middleware   (baris 43-61)  → Verifikasi Basic Auth token
├── POST /api/auth/masuk      (baris 67-74)  → Login
├── POST /api/auth/keluar     (baris 79-81)  → Logout (no-op)
├── GET  /api/auth/saya       (baris 85-87)  → Cek sesi
├── GET  /api/stok            (baris 90-94)  → Ambil semua bahan
├── POST /api/stok            (baris 97-108) → Tambah bahan
├── PUT  /api/stok/:id        (baris 111-121)→ Edit bahan
├── DELETE /api/stok/:id      (baris 124-128)→ Hapus bahan
├── POST /api/stok/:id/catat  (baris 131-151)→ Catat transaksi
└── Export & Listen           (baris 155-159)→ Vercel export + local listen
```

Penjelasan Middleware `harusLogin`:

```
javascript
// 1. Ambil header Authorization
// 2. Decode token dari base64 → "username:password"
// 3. Query ke collection users untuk verifikasi
// 4. Jika valid → simpan data pengguna di req.pengguna
// 5. Jika tidak valid → kembalikan 401 Unauthorized
```

Penjelasan Endpoint Catat Transaksi (`POST /api/stok/:id/catat`):

```
javascript
// 1. Ambil data bahan dari database berdasarkan ID
// 2. Jika tipe 'keluar' dan jumlah > stok → tolak (stok tidak cukup)
// 3. Hitung stok baru: masuk → stok + jumlah, keluar → stok - jumlah
// 4. Update stok di database
// 5. Kembalikan data bahan terbaru + data transaksi
```

Export untuk Vercel:

```
javascript
// Jika dijalankan langsung (node server.js) → listen di PORT
// Jika di-import oleh Vercel → hanya export app
if (require.main === module) {
  app.listen(PORT, () => console.log('Server berjalan...'));
}
module.exports = app;
```

> IMPORTANT
>
> **Yang perlu dilengkapi:**
>
> 1. **Screenshot** untuk setiap fitur di Bagian 1 — buka website, jalankan setiap fitur, lalu ambil screenshot
> 2. **Nama anggota dan NPM** di bagian Cover
> 3. Export dokumen ini ke **PDF** sesuai format pengumpulan